Advanced Algorithms

Assignment 2

CJ Individual Report

```cpp
#include <random>

#include "modarith.hpp"
#include "primality.hpp"

bool fermat(u64 n, TestParams& p) {
    p.rounds_used = 0;
    if (n < 2) {
        return false;
    }
    if (n < 4) {
        return true;
    }
    if ((n & 1) == 0) {
        return false;
    }

    std::uniform_int_distribution<u64> pick(2, n - 2);
    for (int i = 1; i <= p.rounds; i++) {
        p.rounds_used = i;
        if (powmod(pick(p.rng), n - 1, n) != 1) {
            return false;
        }
    }
    return true;
}
```

My algorithm is the Fermat Primality test. This is a probabilistic algorithm to quickly determine if a number is prime or not. If a number is determined to be composite by the algorithm, it is always composite, error comes from if the algorithm determines a number to be prime. Any composite number could be falsely returned as prime with a quickly decreasing chance the higher the number of repeats. The algorithm is also always incorrect on Carmichael numbers, these are numbers which are composite but a likely to be classed as prime by Fermats algorithm. In terms of time complexity, the algorithm has a theoretical running time of O(k log^3 n) due to the way long integer multiplication is handled, however, because we limit the input to 64 bit integers and C++ can compute multiplication of these integers in one CPU cycle, the real world time complexity is O(k log n). Similarly, the space complexity is theoretically O(n) with n denoting the number of bits, but because we used a fixed amount of bits, the real world space complexity is O(1).

My algorithm was extremely simple. The image above is it in its entirety and there were no bugs directly caused by its implementation (due to the small size). The bug which I encountered was during the design of testing. First runs of batches of numbers ran significantly slower than future runs. This was diagnosed to be because of first time loading of the algorithm into (what we believe was) CPU cache. Once this was found, we instead implemented a warm up run before running any testing. While the difference was small, it made our testing suite slightly more accurate and reproducible.

In future, I would create the testing suite first based on the specifics of the algorithm, and then create the algorithms after. Our approach was to create the algorithms first as well as testing scripts and then merge them later which created a lot more work than if we planned it out beforehand.

One thing I didn’t fully understand about my algorithm was how Carmichael numbers created incorrect results and how these took so long to find and formalise (by Carmichael himself about 200 years later). The underlying maths requires an understanding of how the original algorithm works as well as how a counter-example is generalised to any larger number (which is also a Carmichael number). Normally you would assume an algorithm to determine primes would primarily be inaccurate due to scale of numbers rather than a quirk with how it functions, but Fermat shows the opposite. I was also surprised with how easily Miller-Rabin could fix this with a few checks and modifications to the algorithm.